

ARO: The Short Studies

How a Language Gets Built in Ten Pages

ARO Language Project

March 2026

Contents

ARO: The Short Studies	1
How a Language Gets Built in Ten Pages	1
1. The Core Bet	1
2. Lexical Analysis	2
3. Syntactic Analysis	3
4. Semantic Analysis	4
5. Interpreted Execution	5
6. Event Architecture	6
7. Native Compilation	7
8. The Runtime Bridge	8
9. Dual-Mode Parity	9
10. What We Learned	10

ARO: The Short Studies

How a Language Gets Built in Ten Pages

ARO Language Project · March 2026 · ARO 0.9.x

This is the condensed version of *The Construction Studies*. Every major architectural decision, every important data structure, every hard-won lesson — compressed to what you actually need to understand how ARO works. Read the full book for depth. Read this for the map.

1. The Core Bet

ARO is built on one hypothesis: **expressiveness and predictability are inversely correlated**, and for business-feature code, predictability wins.

The consequence is radical constraint. ARO has exactly **nine statement types**:

#	Statement	What it does
1	<code>AROStatement</code>	The core action-result-object form
2	<code>PublishStatement</code>	Exports a variable across feature sets
3	<code>ForEachLoop</code>	Iterates a collection
4	<code>MatchStatement</code>	Pattern matching
5	<code>RangeLoop</code>	Numeric range iteration
6	<code>WhileLoop</code>	Condition-based loop
7	<code>BreakStatement</code>	Exits a loop
8	<code>RequireStatement</code>	Declares a dependency
9	<code>PipelineStatement</code>	Chained actions via <code>\ ></code> operator

Nine you can write. There is a tenth conformer, `ErrorStatement`, which only a parse failure produces — it holds the message and the skipped tokens so a broken file still yields a statement list an IDE can show. That's it. Python's AST has over 40 statement types. Every statement type you don't have is a parser case, a semantic pass, a code generation branch, and a tool integration you don't have to write. The constraint propagates as simplification through the entire stack.

Every action is classified by data flow direction — not as documentation, but as an enforced type:

Role	Direction	Examples
<code>request</code>	External → Internal	Extract, Retrieve, Fetch, Probe, Pull, Clone
<code>own</code>	Internal → Internal	Compute, Validate, Compare, Create, Stage, Checkout
<code>response</code>	Internal → External	Return, Throw
<code>export</code>	Internal → Global	Store, Emit, Log, Send, Publish, Commit, Push, Tag

The role determines valid prepositions, what bridge functions the compiler calls, and what optimizations are legal.

The error philosophy: ARO code is the error message. When `Retrieve the <user> from the <user-repository>` fails, the runtime reports exactly that sentence back. No stack traces needed — the statement *is* the description of what failed.

2. Lexical Analysis

The lexer is a single-pass state machine that makes one unusual choice: **articles and prepositions are first-class token types**, not noise to be filtered.

"Extract the <user> from the <request: body>."

↓

```
[VERB:"Extract"] [ARTICLE:"the"] [RESULT_START:"<"]
[IDENT:"user"] [RESULT_END:">"] [PREP:"from"]
[ARTICLE:"the"] [OBJECT_START:"<"] [IDENT:"request"]
[COLON:":"] [IDENT:"body"] [OBJECT_END:">"] [DOT:"."]
```

Why keep articles? Because `Extract a <user>` and `Extract the <user>` carry slightly different semantics in English, and stripping them would lose that signal for future tooling. The parser filters them for now; the door stays open.

The hard problems:

String interpolation — `"Hello, ${<name>}!"` forces the lexer to recursively lex the embedded expression, yielding a token sequence of string parts and inner tokens that the parser reassembles. It works, but it's the messiest part of the lexer.

Regex vs. division — `/pattern/` and `7 / 2` are lexically identical at the `/`. The lexer resolves the ambiguity with one token of backwards context plus one character of lookahead: after an identifier, after a `.`, or when the next character is whitespace, it's division; otherwise it tries to scan a regex and backtracks if there is no closing `/`. This works for all real-world cases but is technically a heuristic.

String literals — two kinds, selected by the quote character alone. `"..."` processes escapes and may span lines; `'...'` is raw, and only `\` is special. There is no `r` prefix and no triple-quote form: `"""` was removed and now produces a diagnostic naming its replacement.

Source locations — every token carries line, column and byte offset. This costs memory but pays off when error messages need to point at the right character.

3. Syntactic Analysis

The parser is a hybrid: **recursive descent for statements, Pratt parsing for expressions**.

Recursive descent is the right choice for statements — a leading-keyword dispatch table routes each one to its parser, with every unclaimed token falling through to an action statement. Adding a statement form that starts with a new keyword is one table entry.

Pratt parsing handles expressions. The insight is that each token has a *binding power* — how tightly it grabs its neighbors. Addition is weaker than multiplication, which is weaker than unary negation, which is weaker than function call. Parse at the right power level and precedence falls out automatically.

Precedence Level	Operators
1 (lowest)	<code>or</code>
2	<code>and</code>
3	<code>==, !=, is, is not, contains, matches</code>
4	<code><, >, <=, >=</code>
5	<code>+, -, ++</code> (concat)
6	<code>*, /, %</code>
7	prefix <code>-</code> , prefix <code>not</code>
8 (highest)	<code>., []</code>

`not` appears once, at level 7, and it is prefix-only — not an infix operator at some lower level as well. So it binds tighter than any comparison: `not <n> >= 3` groups as `(not <n>) >= 3` and dies with *Cannot convert Bool to number*. Parenthesize when you mean to negate a comparison.

The core statement shape that nearly everything reduces to:

```
Action [article] <Result[:qualifier]> preposition [article] <Object[:qualifier]>.
```

`<Result:qualifier>` is a `QualifiedNoun` — the most important structural unit in ARO. It shows up everywhere:

Field	Type	Example
<code>base</code>	<code>String</code>	<code>"user"</code>
<code>typeAnnotation</code>	<code>String?</code>	<code>"profile.contact.email"</code>
<code>asType</code>	<code>String?</code>	the <code>as <Type></code> result annotation
<code>isLiteralQualifier</code>	<code>Bool</code>	true for <code><file: "data.json"></code>
<code>specifiers</code>	<code>[String]</code> (computed)	<code>["profile", "contact", "email"]</code>

`<user: profile.contact.email>` resolves to base `user` with a dotted property path that `specifiers` splits. The path is dot-separated, not colon-chained — `<user: profile: contact: email>` is a syntax error.

The last two fields are scars from one mistake made twice: a single slot doing two jobs. The qualifier selects an *operation* (`<n: length>`) while `as` requests a *result type*; collapsing them made `Compute the <m: length> as Integer from <s>. return the string instead of its length.` And a quoted qualifier is a value, never a property path — without `isLiteralQualifier`, `<file: "data.json">` resolved to the path `data`.

Error recovery: when the parser hits something unexpected, it scans forward to the next `.`, `}` or `<` and continues, leaving an `ErrorStatement` holding the skipped tokens where the broken statement was. A single syntax error doesn't abort the whole file — you get all the errors in one pass.

But recovery is a *choice*, and the caller makes it. `Parser.parse(source, diagnostics:)` recovers; `Parser.parse(source)` throws. A caller with nowhere to read errors from cannot otherwise tell a file that failed to parse from a file with no feature sets — both are a `Program` with zero feature sets, and reading a broken file as empty is how a graph diff once reported 144 feature sets rewritten.

4. Semantic Analysis

`SemanticAnalyzer` is under 400 lines because it conducts rather than works. Five specialist analyzers — `DataFlowAnalyzer`, `CodeQualityValidator`, `CollectionOpValidator`, `EventAnalyzer`, `UserActionAnalyzer` — own the checks; the conductor owns the order, and the order is load-bearing in both directions:

1. **Duplicate names**, then the **user-action registry**, so call sites can be validated against it
2. **Per feature set**: symbol table, data flow, code quality, collection ops; publish to the global registry
3. **Dependency verification** against that registry, enforcing business-activity boundaries
4. **Circular event chains**, then **orphaned emissions** — both need every feature set analyzed first
5. **User-action call sites**, then unavoidable recursion — last, so the duplicate-name complaint precedes the complaints that follow from it

VerbSets are the semantic analyzer’s classification tool. Every verb gets assigned to exactly one set, and that set determines how the executor treats it:

Set	Verbs (examples)	Effect
testVerbs	then, assert	Fall through so the test actions see bindings
requestVerbs	call, request, fetch, retrieve, parse, exists	External invocation — always execute
updateVerbs	update, modify, set, configure	Always execute; they handle rebinding themselves
createVerbs	create, make, build	Execute when specifiers are present
responseVerbs	write, read, store, save, log, send, emit	Result must not be rebound to the expression value
serverVerbs	start, stop, keepalive, schedule, sleep	Force execution even with a literal object

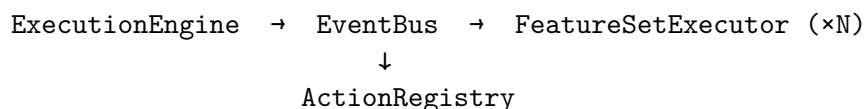
Eleven sets in all — there is no **storeVerbs**; **store**, **save** and **persist** live in **responseVerbs**. They answer one question the executor asks per statement: may the expression fast path be taken, or must the action actually run? Nearly every entry carries a bug number where the answer was “yes” and should have been “no”, and every one of those bugs produced a *wrong answer* rather than a crash — **Exists the <flag> for “./path”** bound the path string instead of the boolean; **Delete the <gone> from “./f.txt”** deleted nothing and reported [OK].

The sets live in **VerbSets.swift**. The interpreter consults them per statement; binary mode emits direct action calls and has no such decision to make, so for the compiler this is a canonical vocabulary reference rather than a runtime dependency. The parity win came from deleting a duplicate table — and it only covers the lists you actually share. The framework variables cleared between statements are a second per-statement list that never got the same treatment, and they have drifted into a wrong answer (GitLab #552).

Immutability is the rule with the most impact on users: once a variable is bound in a feature set, it can’t be rebound. The only escape is renaming — compute a new name. This feels annoying until you debug a feature set and realize you always know exactly what a variable contains at any point.

5. Interpreted Execution

The interpreter is built around three collaborating components:



ExecutionEngine is a Swift actor. It loads the program, runs ten handler-registration passes against the event bus, and fires **Application-Start**. After that it steps back and lets events drive

everything, draining until quiescent — and warning if the pending count stops decreasing rather than hanging.

FeatureSetExecutor runs one feature set. It is a `final class`, not an actor. For each statement it: 1. Opens a fresh statement scope and clears the framework variables 2. Builds a `ResultDescriptor` and `ObjectDescriptor` from the AST node 3. Looks up the verb in `ActionRegistry` 4. Calls `action.execute(result:object:context:)` — which may return a future rather than a value 5. Checks if the action set a response (short-circuit) or threw

ActionRegistry maps verb strings to action implementations: 71 built-in actions claiming about 130 verbs, assembled from eleven action modules rather than registered one at a time. It is lock-guarded rather than actor-isolated, because it is a table read once per statement from call paths — including the C bridge — that cannot `await`. `aro actions` prints the live list.

Deferred execution (ARO-0088) is the part that changes how the loop reads. A statement *starts* where you wrote it; the program waits for it at the first read of its result. Two independent two-second requests in one feature set cost about two seconds. Deferral is an allowlist of thirty value-producing verbs, not a heuristic: effects (`Log`, `Return`, `Emit`, `Publish`, ...) always run at their own statement and force what they read, so observable output stays in source order. `Sleep` is deliberately excluded — the delay *is* the effect. `ARO_NO_DEFER=1` turns the whole thing off, which is the fastest way to find out whether a bug is about ordering.

ExecutionContext is what actions see. A flat protocol that hides whether you're running in the interpreter, a compiled binary, or a test:

Method group	What it does
<code>resolve, require, bind</code>	Variable read/write
<code>service</code>	Access HTTP, file, socket services
<code>repository</code>	CRUD storage
<code>setResponse, getResponse</code>	Track the short-circuit response
<code>emit</code>	Fire events

The short-circuit pattern: as soon as a `Return` or `Throw` action runs, it sets the response on the context. The executor checks after every statement and stops early. This is how `Validate the <user>` can abort the rest of the feature set — no exceptions, no special control flow.

6. Event Architecture

Everything in ARO is event-driven. HTTP requests arrive as events. File changes arrive as events. Domain events travel between feature sets as events. The `EventBus` is the central router.

Handler types are registered by naming convention — the classifier is one pass that tests the business activity string against ten patterns, top to bottom:

Pattern	Triggered by
<code>... Watch: ...</code>	A watched path or repository

Pattern	Triggered by
{field} StateObserver, StateTransition Handler	Accept-action state transitions
KeyPress Handler	Keyboard input (with where guards)
Socket Event Handler	TCP socket events
WebSocket Event Handler	WebSocket lifecycle
File Event Handler	File system changes
NotificationSent Handler	Notify action
{repo-name} Observer	Repository store/update/delete
{repo-name} Evicted Handler	Repository eviction
{Name} Handler (none of the above)	Custom domain event

The order *is* the specification: **Socket Event Handler** must be tested before the generic "**Handler**" rule, whose exclusion list is what stops it claiming everything.

Two things are commonly assumed to be on that list and are not. HTTP routes are matched by `operationId` from `openapi.yaml`, never by name pattern. And ARO-0081 **Action** feature sets are called directly as `Application.<Name>` — they never touch the bus.

StateGuards let handlers filter on event payload fields:

```
(Send Welcome: NotificationSent Handler) when <age> >= 18 {
    (* only fires when the notified user is 18+ *)
}
```

The guard is evaluated before the handler body runs. If it fails, the handler is silently skipped.

publishAndTrack is how the event bus ensures handlers complete before the program moves on. Every dispatched handler increments an in-flight counter. `awaitPendingEvents(timeout:)` does not spin: it races a `withCheckedContinuation` — registered in a table and resumed the instant the counter reaches zero — against a timeout sleep that removes that specific continuation. Whichever wins, the return value says which. This prevents race conditions between **Application-Start** and its handlers, a real bug we hit early.

DomainEvent co-publishing is the bridge between interpreter and binary mode. Typed Swift events (like `FileCreatedEvent`) work for the interpreter. Compiled binaries can't receive those — they need C-callable registration functions. The solution: every action that fires a typed event *also* publishes a `DomainEvent` (a plain string type + `[String: Sendable]` dictionary) to the same bus. Binary handlers register via C functions that subscribe to these `DomainEvents`.

7. Native Compilation

The compiled path takes the same AST and turns it into a native binary instead of interpreting it.

`.aro` files → Parser → AnalyzedProgram → LLVMCodeGenerator → LLVM IR → `llc` → `.o` → Linker → binary

Swift-LLVM: ARO uses the Swift-LLVM wrapper around LLVM's C API. The original implementation used textual LLVM IR — strings concatenated together. The problem: type errors were only caught when `llc` tried to compile the output. With Swift-LLVM, IR is built from typed

Swift objects (`Function`, `BasicBlock`, `GlobalVariable`), and `verifyModule()` catches structural errors before any text is emitted. The cost: LLVM 20 as a build dependency.

What gets generated: each feature set becomes an LLVM function. The function: 1. Allocates a context handle (opaque pointer to a `RuntimeContext`) 2. Allocates descriptor structs for each statement’s result and object 3. Calls the appropriate `aro_action_*` C function 4. Checks the return value and jumps to `error_exit` on failure 5. Frees the context and returns

String constants are collected in a first pass and emitted as global LLVM constants. The code generator then references them by pointer — no string allocation at runtime.

Control flow maps directly to LLVM basic blocks: - **when** guard → branch instruction to skip block - **match** → chain of comparisons, each branching to its case block - **for-each** → loop with **gep** to advance through the array - **while** → back-edge to the condition block

Event handler registration happens at program startup: `LLVMCodeGenerator` emits calls to C-callable registration functions (`aro_runtime_register_notification_handler`, `aro_runtime_register_state_transition_handler`, etc.) that subscribe to `DomainEvents` and call the compiled function pointer when the event fires.

8. The Runtime Bridge

The compiled binary can’t call Swift directly — the C ABI boundary forbids it. Everything flows through `Sources/ARORuntime/Bridge/`, fourteen files exposing **248 @cdecl functions**. (There is no `ARORuntime` module and no `RuntimeBridge.swift`: the C surface was folded into `ARORuntime`, now a static library exporting C symbols, and the old monolith was split by concern.)

LLVM Binary → @cdecl functions → Swift runtime
(C types) (C ABI boundary) (async, actors, closures)

Three handle types cross the boundary as opaque `UnsafeMutableRawPointer`:

Handle	What it wraps	Used for
<code>AROCRuntimeHandle</code>	The <code>Runtime</code> , the context table, the event-loop group	Shared runtime state
<code>AROCContextHandle</code>	One <code>RuntimeContext</code> plus the services it must keep alive	Per-execution variable space
<code>AROCValue</code>	A lock-guarded <code>any Sendable</code>	Returning values to C

Descriptors do *not* get a handle. `ResultDescriptor` and `ObjectDescriptor` cross as stack-allocated C structs the code generator fills per statement. Only things with lifetimes get handles.

The synchronous bridge problem (and how it was solved): @cdecl functions can’t be `async`, but Swift runtime actions are `async`. ARO solves this with an `AROFuture` — `aro_action_*` returns immediately with a future handle, and force points (the value-accessors, the effectful verbs) block the C-bridge pthread on the future’s `DispatchGroup` until the result is ready.

Two details carry the fix, and they are separable. The futures’ tasks run on a custom `TaskExecutor` over GCD’s *elastic* global queue, not Swift’s fixed-size cooperative pool, so a blocked pthread

cannot starve the work that would unblock it. And the handover is a `DispatchGroup` rather than a `DispatchSemaphore` because `group.wait()` wakes every current and future waiter where `signal()` releases exactly one — the right primitive when any number of consumers may force the same value. Semaphores remain at the older one-shot call sites, where a single caller waits for a single answer.

Cascading event chains that previously risked deadlock now finish in milliseconds.

Platform types: `Bool` is not the same on macOS and Linux at the C boundary. macOS passes it as 1 byte; Linux sometimes expects 4 bytes. All boolean results cross the bridge as `Int32` and are converted at each side.

9. Dual-Mode Parity

Running `aro run` and running the compiled binary should produce identical results. In practice, two subsystems diverge:

Expression evaluation has two implementations — `Core/ExpressionEvaluator.swift` for the interpreter and `evaluateBinaryOp()` in `Bridge/RuntimeExecutionBridge.swift` for the binary. They have to be kept in sync manually. The most common drift: integer division. The current one: the compiler’s `ExpressionSerializer` emits `$unary` and the bridge’s evaluator knows only `$lit`, `$var`, `$binary` and `$interpolated`, so a non-constant `not <flag>` in a compiled guard falls through.

Expression	Interpreter (before fix)	Binary	After fix
<code>7 / 2</code>	3.5 (promoted to Double)	3 (truncated)	3 (both)
<code>80 / 3</code>	26.666...	26	26 (both)
<code>7.0 / 2</code>	3.5	3.5	3.5 (both)

The fix: interpreter integer division now truncates, matching the binary. Visible behavior change, but consistent.

Event dispatch is architecturally different between modes. The checklist for adding any new event type:

1. Add `DomainEvent` co-publish after the typed event publish
2. Document the payload schema in a `// DomainEvent payload: comment`
3. Add the `@cdecl` registration function in `Bridge/RuntimeEventRecordingBridge.swift`
4. Declare the extern in `LLVMC/LLVMExternalDeclEmitter.swift`
5. Detect the business activity in `LLVMCodeGenerator.registerEventHandlers`
6. Spread guard fields into the payload if the handler has a `when` condition
7. Add an example with `mode: both` in the test suite

The `mode: both` test directive is the defense mechanism, and the reason it works is that `both` is the **default** — an example opts *out* of parity testing, never in, and an unrecognised mode value resets to `both` rather than disabling the check. Of 102 examples with a `test.hint`, four opt out, and only one of those four is a genuine capability gap.

It is a defense, not a proof. It catches divergence in behaviour some example exercises. GitLab #552 — a compiled binary reusing the previous statement’s `with { separator: "-" }` — slipped

through because no example calls the same qualifier twice, once with a modifier and once without.

10. What We Learned

The things that worked:

- Uniform syntax makes tooling cheap. One statement shape means one parser, one formatter, one AST walker.
- Nine statement types kept every backend small. Every new execution target (interpreter, LLVM, future transpiler) starts with nine cases.
- `mode: both` testing caught divergence that would otherwise be invisible for months — because `both` is the default, so an example has to opt out.
- VerbSets as a shared module eliminated the drift between interpreter and compiler verb classification.
- Deferral as an allowlist rather than a heuristic. An unknown verb runs eagerly, which is always correct and sometimes slow; the reverse default would be sometimes fast and occasionally wrong.

The things that didn't:

- SwiftNIO can't initialize in compiled binaries — Swift type metadata isn't set up when the runtime starts from LLVM-compiled code, and the crash lands in `_swift_allocObject_` with a null metadata pointer. The answer was a native BSD socket server, which has since grown contract routing, WebSocket and streamed request bodies. So the cost is not “no HTTP in binaries”; it is two server implementations to keep in step, and one of them can't do chunked transfer encoding.
- `@cdecl` can't be `async`. Two designs later — an elastic executor and a `DispatchGroup` — it works and is no longer fragile, but the boundary is still where the hardest bugs live.
- Business activity scope confused everyone. Variables visible only within the same activity string is neither global scope nor lexical scope. Users expected one or the other.
- Prepositions carry semantic meaning (`from` = source, `into` = destination, `with` = modification) but the distinctions are too subtle. Developers guess wrong and get cryptic errors.
- Two hand-maintained lists that must agree will not. `VerbSets` fixed one such pair by deleting the duplicate; the framework variables cleared between statements are the pair that never got the same treatment, and they now disagree in a way that changes results (GitLab #552).

The evolution in one table:

Version	What changed
0.1	5 statement types, text-based LLVM IR, no binary mode
0.3	Added RangeLoop, WhileLoop, BreakStatement → 8 types
0.5	VerbSets extracted as shared module; dual-mode parity project starts
0.6	Swiftly-LLVM replaces text IR; <code>verifyModule()</code> catches IR errors early
0.7	DomainEvent co-publishing; <code>mode: both</code> becomes the default

Version	What changed
0.7.1	Integer division parity; KeyboardService binary mode
0.8.0	PipelineStatement (9 types); KeyPress/WebSocket handlers; package manager (<code>aro add</code> / <code>aro remove</code>); LSP server; terminal UI subsystem
0.9.x	Deferred execution as the default, not a flag (ARO-0088); user-defined actions (<code>Application.<Name></code> , ARO-0081); native Git via libgit2 (ARO-0080); streaming request bodies with per-route limits, in both modes (ARO-0090); one string delimiter — plain <code>"..."</code> spans lines, <code>"""</code> removed; MCP server (<code>aro mcp</code>); <code>aro ask</code> ; the Jupyter kernel (ARO-0091)

For the next implementer:

Keep the statement count small. It's the most impactful architectural decision and the hardest to change later. Every statement type you add is a permanent tax on every future pass, every future tool, every future backend.

Test both execution paths from day one. Divergence accumulates silently. By the time you notice, it's deep in the codebase — and make the parity check the default, because a check you have to remember to add is a check you will forget.

Any fact stated in two places will disagree. Not might: will. The only durable fix is one place plus a test that fails when a second one appears.

Co-publishing is not elegant, but it works. A unified event system would be cleaner. In practice, unifying deeply integrated subsystems mid-project costs more than bridging them.

The constraint is the feature. ARO programs are predictable and auditable because the language doesn't let you write anything else. That's not a limitation — it's the entire point.

For the complete treatment — all diagrams, all implementation details, all appendices — see The Construction Studies.